

Validations & Transformations — Lecture Notes

1. Where Validations Fit in the Backend Architecture

A typical backend has three layers:



Validations and transformations happen at the entry point of the Controller layer — after route matching, before any business logic or DB calls are made.

Why so early? Garbage in → garbage out. Catching bad data at the boundary prevents propagation deep into the system and avoids cryptic errors (like a `500 Internal Server Error` from a DB type constraint violation instead of a clean `400 Bad Request`).

2. What is Validation?

Ensuring all incoming data — JSON body, query params, path params, headers — is in the exact format the API expects before any logic runs.

If validation fails → return `400 Bad Request` immediately with a descriptive error. No service call, no DB call.

What validation covers: presence of required fields, correct data types, value constraints, structural patterns, logical consistency.

3. Types of Validation

3.1 Type Validation

The most basic: does the field have the correct data type?

Field	Expected	Received	Result
age	number	"25" (string)	✗ Error
tags	string[]	[1, 2] (number[])	✗ Error
active	boolean	"false" (string)	✗ Error

Works recursively — nested objects and array elements can each have their own type constraints.

3.2 Syntactic Validation

Does the value satisfy a required **structural pattern**, regardless of semantic meaning?

Field	Validation	Example Failure
email	Must match <code>local@domain.tld</code> pattern	"notanemail"
phone	Country code + digit count pattern	"123"
date	Must be valid <code>YYYY-MM-DD</code> format	"2025-13-45"
name	Length between 5–100 characters	"ab"

These are purely structural — a syntactically valid email doesn't mean the inbox exists.

3.3 Semantic Validation

Does the value **make sense in the real world** — given its context and business rules?

Field	Rule	Example Failure
dateOfBirth	Cannot be in the future	2026-01-01
age	Must be between 1–120	430
endDate	Must be after <code>startDate</code>	<code>endDate < startDate</code>

Semantic validation catches logically impossible or unrealistic values that pass syntactic and type checks.

3.4 Complex / Cross-Field Validation

Validation rules that span **multiple fields** or depend on the value of another field.

Examples:

- `passwordConfirmation` must exactly equal `password`
- `password` must be at least 8 characters
- If `married = true`, then `partnerName` is required (otherwise optional)

```
{
  "password": "secret1",           // must be ≥ 8 chars
  "passwordConfirmation": "secret1", // must equal password
  "married": true,
  "partnerName": "Alice"          // required only when married = true
}
```

These dependencies can't be expressed field-by-field; the validation logic needs access to the full payload.

4. What is Transformation?

Converting incoming data into the format expected by the service layer — before or after validation, as part of the same pipeline.

Why it's necessary

Query parameters are **always strings** when they arrive at the server, regardless of what the client intended. A value like `?page=2&limit=20` arrives as `{ page: "2", limit: "20" }`. Validating these as numbers without first casting them will fail — the type check sees strings.

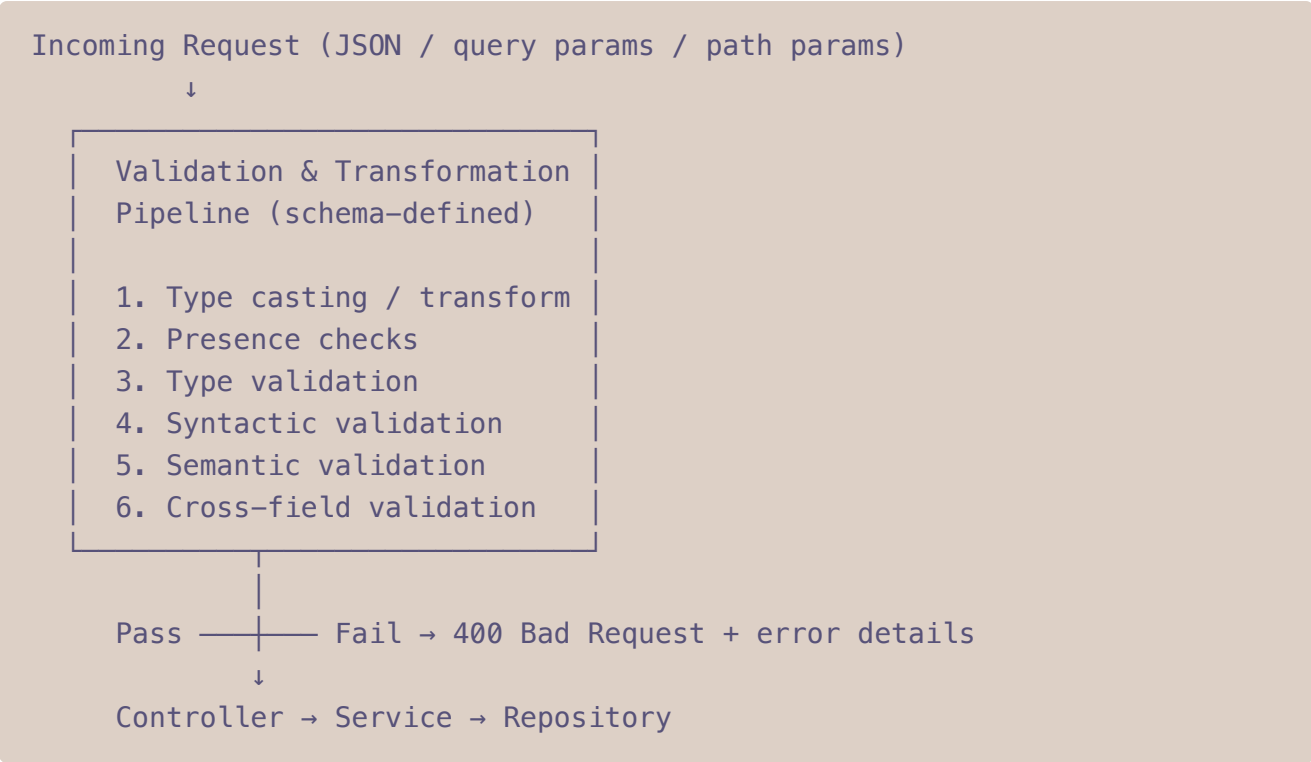
Transformation → cast string `"2"` to number `2` → then validate `> 0 && < 500`.

Common transformation operations

Operation	Example
Type casting	<code>"20"</code> → <code>20</code> (query param → number)
Normalization	<code>"Test@GMAIL.COM"</code> → <code>"test@gmail.com"</code> (email to lowercase)
Sanitization	Strip leading/trailing whitespace from strings
Formatting	<code>"9876543210"</code> → <code>"9876543210"</code> (prepend <code>+</code> to phone)
Date normalization	<code>"2025-11-05"</code> → ISO 8601 format

5. The Validation & Transformation Pipeline

Validations and transformations are combined into a **single reusable pipeline** at the controller entry point — typically a middleware or utility function configured with a schema.



Keeping everything in one place means you never have to hunt across layers to understand what an API expects.

6. Frontend vs Backend Validation

A critical distinction that is often misunderstood:

	Frontend Validation	Backend Validation
Purpose	User experience — immediate feedback	Security & data integrity
When it runs	Before the API call	After the API call reaches the server
Can be bypassed?	Yes — Postman, curl, direct API calls	No — always runs
Required?	Recommended, not mandatory	Mandatory

Never rely on frontend validation for security. A server has many clients — a web app, a mobile app, Postman, curl, other servers. Only the backend can enforce constraints unconditionally.

Bad assumption: "Our React form validates the email, so the backend doesn't need to"
Reality: Anyone can call `POST /api/users` with a raw HTTP request and bypass the form entirely

Rule: Design backend validations independently of whatever the frontend does. Frontend validation is a UX courtesy; backend validation is a security requirement.

7. Summary

Concept	Definition	Location
Type validation	Check data types match expected types	Entry point of controller
Syntactic validation	Check value matches a structural pattern (email, phone, date)	Entry point of controller
Semantic validation	Check value is logically meaningful (DOB not in future, age ≤ 120)	Entry point of controller
Complex validation	Cross-field rules (password match, conditional required fields)	Entry point of controller
Transformation	Cast/normalize/format data before passing to service layer	Entry point of controller

Quick Revision Checklist

- ☐ Validations run at the controller entry point — after route match, before any service/DB calls
- ☐ Failed validation → `400 Bad Request` with descriptive error; never let bad data reach the service layer
- ☐ Type validation: correct data type (string, number, boolean, array)
- ☐ Syntactic validation: structural pattern match (email, phone, date format, string length)
- ☐ Semantic validation: logical correctness (DOB not in future, age within realistic range)
- ☐ Complex validation: cross-field rules (password confirmation, conditional required fields)
- ☐ Transformation: cast types (query params are always strings → cast to number), normalize (email → lowercase), format (phone → add `+`)
- ☐ Validation + transformation live in one pipeline → single source of truth for input contract
- ☐ Frontend validation = UX only; Backend validation = security + data integrity; **never substitute one for the other**

- ☐ Validation errors double as implicit API documentation — empty payload returns all expected fields